

```
cmake --build build/  
./build/Debug/task03.exe
```

Aufgabe03

3.1) Nutzen der Farbattribute.

Dafür muss zuerst bei der Erstellung der Vertexes die Farbe als Attribut hinzugefügt werden. Im Vertex-Shader muss dann das Farbattribut als Input und Output deklariert werden und an den Fragment-Shader weitergegeben werden. Im Fragment-Shader muss dann die Farbe als Input deklariert und mit *outColor* verbunden werden.

3.2) Zylinder, Quader, Kugel

Zylinder

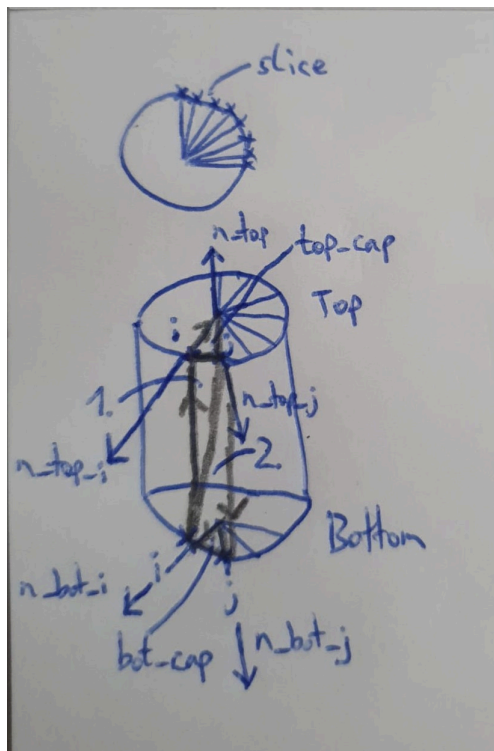


Figure 1: Zylinder

1. Vorberechnung der Punkte auf dem Ring für Top und Bottom, da diese für den Mantel und die Deckel wiederverwendet werden. Anzahl der Ringpunkte wird basierend auf der Anzahl der Slices berechnet.
2. Berechnung der beiden Mittelpunkte und Normalenvektoren für die beiden Deckel. Die Normalenvektoren zeigen nach oben für den oberen Deckel und nach unten für den unteren Deckel.
3. Pro Durchgang in der Schleife werden 1 Dreieck für den Deckel und 1 Dreieck für den Boden berechnet, sowie 2 Dreiecke für den Mantel (bilden Quad). Die Normalenvektoren für die Seitenfläche werden basierend auf der Position der Punkte auf dem Ring berechnet (betrachtet vom jeweiligen Mittelpunkt aus). Die UV-Koordinaten gehen von (0,0) unten links bis (1,1) oben rechts.

Quader

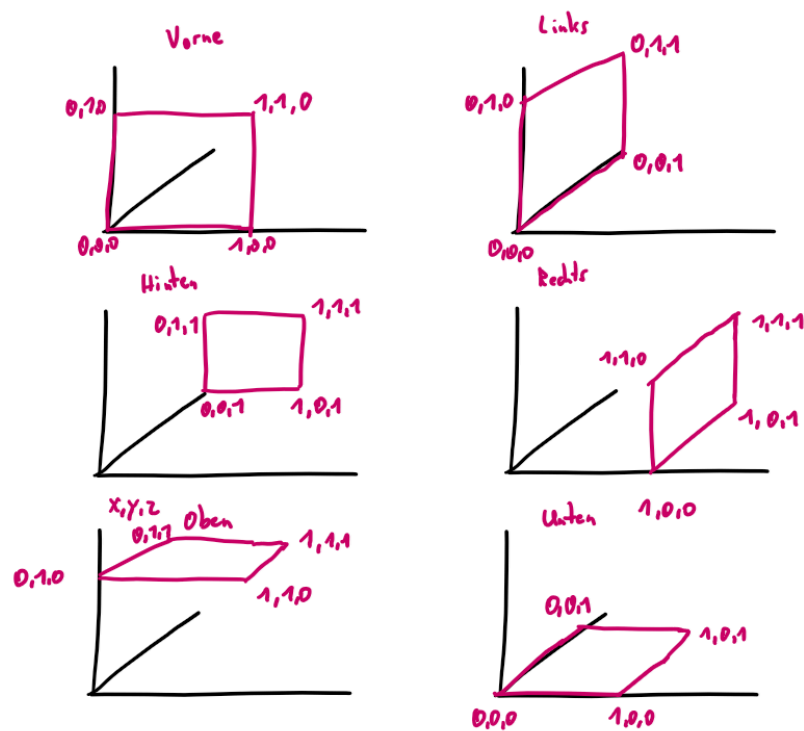


Figure 2: Quader

1. Hartcodiert die 8 Ecken des Quaders eines Einheitswürfels mit den Normalenvektoren definiert. Die Normalenvektoren stehen auf den Eckpunkten und zeigen in die Achsenrichtung der Fläche (+X, -X, +Y, -Y, +Z, -Z).

Kugel

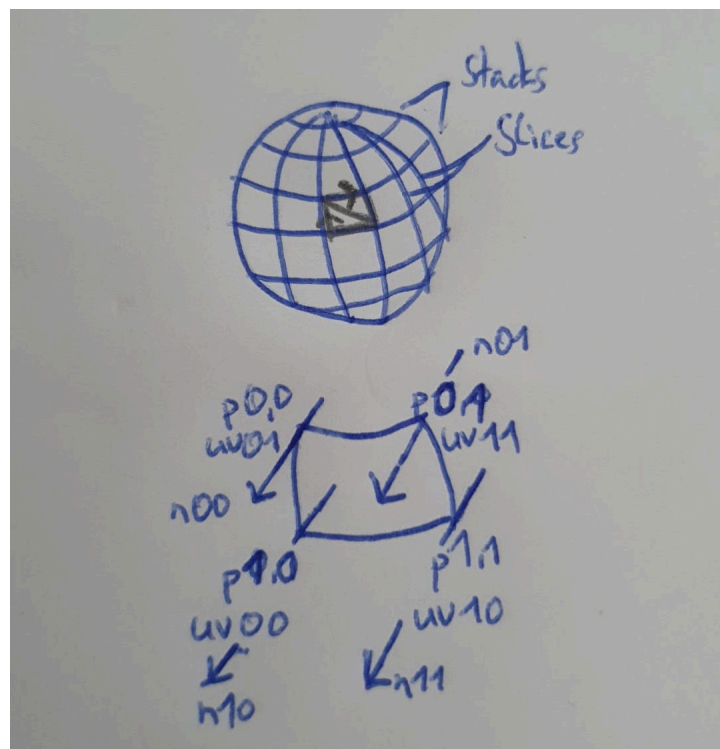


Figure 3: Kreis

1. Man unterteilt die Kugel in Ringe (Stacks) und Segmente (Slices). Die Anzahl der Stacks und Slices bestimmt die Auflösung der Kugel.
 - Für jeden Stack wird ϕ_0 und ϕ_1 berechnet, die die vertikalen Winkel für die Ringe definieren.
 - Für jeden Slice wird θ_0 und θ_1 berechnet, die die horizontalen Winkel für die Segmente definieren.
2. Basiert auf den berechneten Winkeln werden mithilfe von Sinus und Cosinus die 3D-Koordinaten der Punkte auf der Kugeloberfläche berechnet.
3. Die vier berechneten Punkte bilden zwei Dreiecke, die zusammen ein Quad ergeben. Die Normalenvektoren für die Dreiecke sind identisch mit den Positionen der Punkte, da die Normalenvektoren bei einer Kugel von ihrem Mittelpunkt nach außen zeigen. Die UV-Koordinaten werden von (0,0) unten links bis (1,1) oben rechts aufgespannt.

Alternativer Ansatz für die Kugel (über Einheitswürfel)

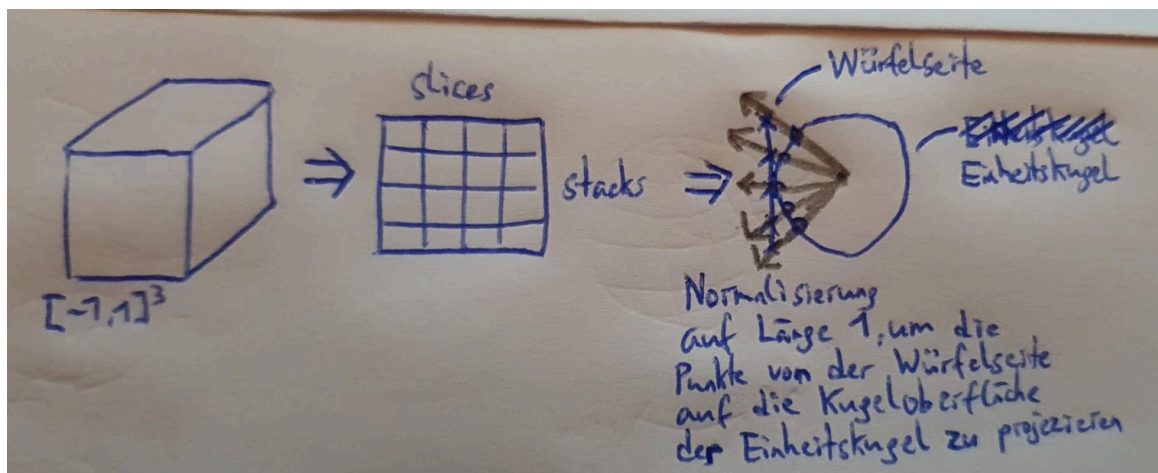


Figure 4: Einheitskugel

1. Man verwendet einen Einheitswürfel mit 8 Ecken als Basis. Jede Seite des Würfels wird in Stacks und Slices unterteilt, um die Auflösung zu bestimmen.
2. Für jeden Stack und Slice werden die 3D-Koordinaten der Punkte auf der Oberfläche des Würfels berechnet. Diese Punkte werden dann normalisiert, um sie auf die Oberfläche einer Kugel zu projizieren.
3. Die Normalenvektoren für die Dreiecke sind identisch mit den Positionen der Punkte, da die Normalenvektoren bei einer Kugel von ihrem Mittelpunkt nach außen zeigen. Die UV-Koordinaten werden von (0,0) unten links bis (1,1) oben rechts aufgespannt.

3.3) Rendern der Geometrien

Alle drei Geometrien (Zylinder, Quader, Kugel) werden auf die gleiche Art und Weise gerendert.

- Zuerst generiert man die Vertex-Daten über die zuvor erklärten Funktionen.
- Dann werden die Vertex-Daten in einem Vertex Buffer Object (VBO) gespeichert und mit einem Vertex Array Object (VAO) verbunden.
- Im VAO werden die Attribute (Position, Normalenvektor, Farbe, UV-Koordinaten) definiert und mit den entsprechenden Attribut-Standorten im Shader verbunden.
- In der Rendering-Loop werden die VAOs gebunden und die Geometrien mit `glDrawArrays` gerendert.

3.4) Debugging der Normalenvektoren

Wie testen Sie, ob Ihre Normalenvektoren korrekt sind und auch korrekt im Shader sind? Nutzen Sie mindestens zwei Möglichkeiten. Dokumentieren Sie Ihr Vorgehen.

Lösung

1. Möglichkeit: Visualisierung der Normalenvektoren

- Normalenvektoren als Linien in der Szene visualisieren. Dazu wird für jeden Vertex eine Linie gezeichnet zwischen seiner Position und einem Punkt auf seiner Normalen. Die Farbe wird basierend auf seiner Ausrichtung berechnet. Wenn die Normalen korrekt sind, sollten sie in der Szene sichtbar sein und die Farben sollten entsprechend der Ausrichtung variieren.
- Farbkennzeichnung: +X (rechts) 1.0 0.5 0.5 Rot -X (links) 0.0 0.5 0.5 Cyan +Y (oben) 0.5 1.0 0.5 Grün -Y (unten) 0.5 0.0 0.5 Magenta +Z (vorne) 0.5 0.5 1.0 Blau -Z (hinten) 0.5 0.5 0.0 Gelb-Braun
- Für die Erstellung der Linien wurde eine Funktion erstellt, die die Normalenvektoren als Linien in der Szene visualisiert. Es wurden zusätzliche Vertex-Daten für die Linien erstellt, die die Start- und Endpunkte der Linien sowie die Farben enthalten. Diese Linien wurden dann mit einem eigenen VAO und VBO gerendert.

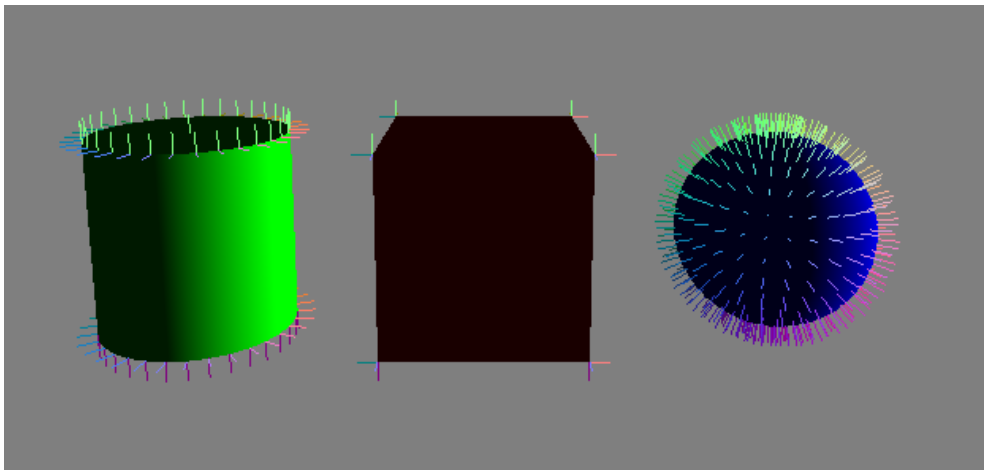


Figure 5: Visualisierung der Normalenvektoren jedes Vertex als Linie mit Farbkennzeichnung der Ausrichtung

2. Möglichkeit: Verwendung von RenderDoc

- In RenderDoc die gerenderten Frames analysieren. In RenderDoc können die Normalenvektoren im Mesh Viewer unter VS Input (Vertex-Shader) inspiziert werden. Zudem kann überprüft werden, ob diese korrekt an den Fragment-Shader weitergegeben werden (VS Out)
- "out_Normal = in_Normal;" zu Vertex-Shader hinzugefügt, um die Normalen direkt an den Fragment-Shader weiterzugeben, und in RenderDoc überprüft, ob die Werte korrekt sind.

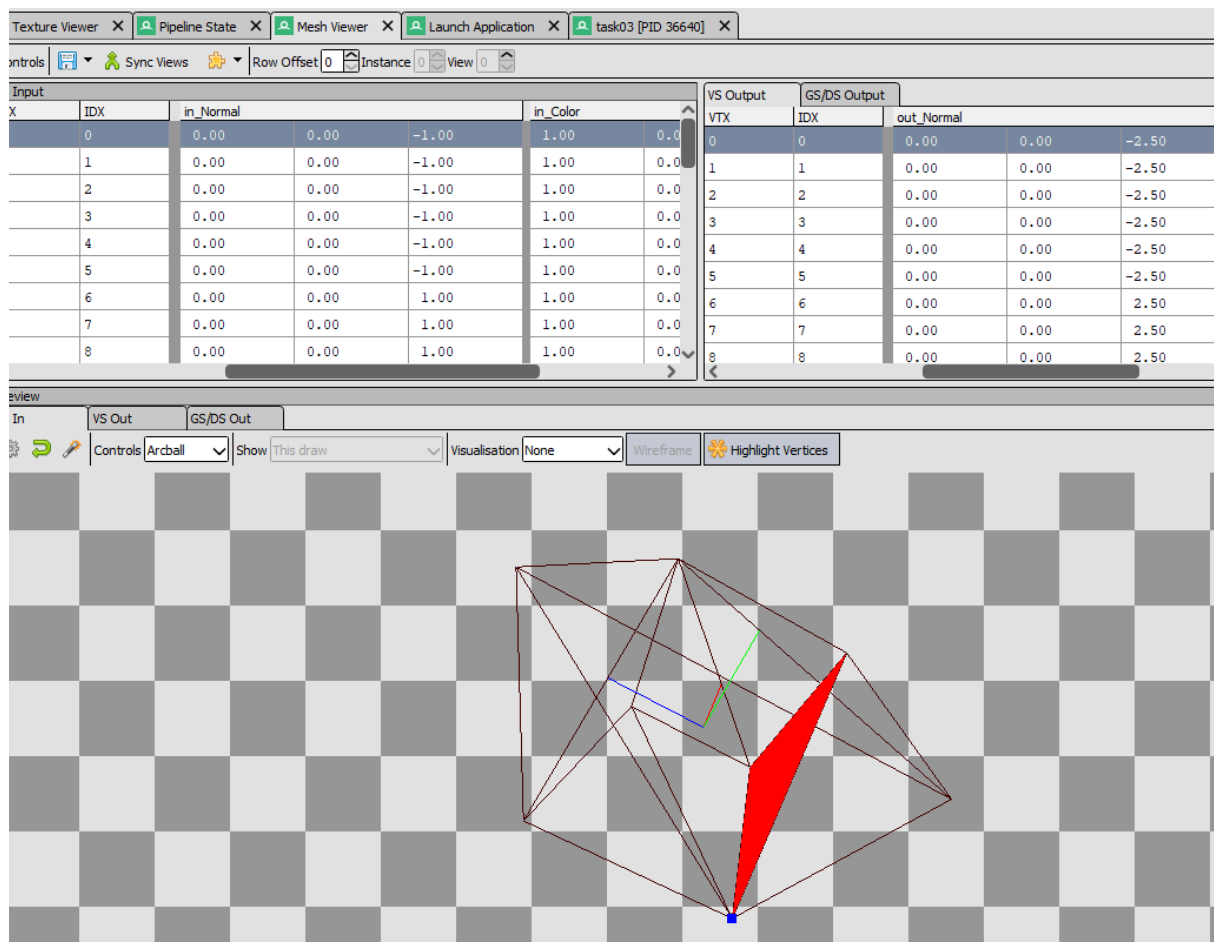


Figure 6: Überprüfung der Normalenvektoren jedes Vertex in RenderDoc unter VS Input (Vertex-Shader) und VS Out (Fragment-Shader)

3.5) Beleuchtung

Recherchieren Sie hierzu das Lambert cosine law. Gibt es bei der Transformation von Modellen hinsichtlich der Normalenvektoren etwas zu beachten? Falls ja, erläutern Sie was das Problem ist und leiten Sie eine geeignete mathematische Lösung her.

Lösung

- Das Lambert cosine law besagt, dass die Intensität des von einer Oberfläche reflektierten Lichts proportional zum Kosinus des Winkels zwischen der Normalen der Oberfläche und der Richtung des einfallenden Lichts ist. Je größer der Winkel zwischen Normalen und Lichtquelle, desto schwächer die Beleuchtung.
- Bei der Transformation von Modellen bei nicht-uniformen Skalierungen können die Normalenvektoren nicht einfach mit der Modellmatrix transformiert werden, da dies zu verzerrten Normalen führt. Das Problem ist, dass die Normalenvektoren orthogonal zur Oberfläche bleiben müssen, aber eine nicht-uniforme Skalierung kann diese Orthogonalität zerstören. Hat ein Tangentenvektor t eine positive x -Komponente, wird eine x -Streckung t nach rechts drehen (z. B. um -18°). Der Normalenvektor n mit negativer x -Komponente wird durch dieselbe x -Streckung nach links gedreht (z. B. um $+18^\circ$). Das Ergebnis ist, dass n nicht mehr orthogonal zu t ist, was zu falschen Beleuchtungsergebnissen führt.

- Deshalb muss die Transformation der Normalenvektoren mit der Inversen Transponierten der Modellmatrix erfolgen, um sicherzustellen, dass die Normalen korrekt transformiert werden und ihre Orthogonalität zur Oberfläche beibehalten wird.
- Die trigonometrische Funktion Cosinus ist 1 bei 0° und 0 bei 90°
- $\text{diffuse} = \max(0, \text{dot}(\mathbf{N}, \mathbf{L})) * \text{lightColor} * \text{objectColor}$
 - Mit $\max(0, \dots)$ werden nur positive Werte berücksichtigt - keine negativen Beleuchtungswerte
 - $\text{dot}(\mathbf{N}, \mathbf{L})$ berechnet den Winkel zwischen Normalenvektor $\mathbf{N}$ und Lichtvektor $\mathbf{L}$
 - lightColor ist die Farbe des Lichts

Implementierung

Vertex-Shader

- Der Normalenvektor mit der Transponierten Inverse der Modellmatrix (*normalMatrix*) transformiert, um sicherzustellen, dass die Normalen korrekt transformiert werden.
- Zusätzlich werden noch die Weltkoordinaten der Vertex-Position berechnet und an den Fragment-Shader weitergegeben, um die Berechnung der Beleuchtung im Fragment-Shader zu ermöglichen.

Fragment-Shader

- $\mathbf{N}$ aus den interpolierten Normalenvektoren berechnet und normalisiert, da der Rasterizer die Normalenvektoren interpoliert und dadurch nicht mehr normalisiert sein können.
- $\mathbf{L}$ wird als Vektor von der Fragment-Position zur Lichtquelle berechnet und ebenfalls normalisiert.
- Der diffuse Beleuchtungsanteil wird aus dem Skalarprodukt von $\mathbf{N}$ und $\mathbf{L}$ berechnet, mit ambient addiert (damit es nicht komplett schwarz ist) und mit der Objektfarbe multipliziert.

3.6) Bewegung der Kamera

Erweitern Sie das Programm, sodass Sie die Kamera mit der Tastatur steuern können. Sie sollten in der Lage sein, die Kamera entlang ihrer Achsen zu verschieben und um einen Winkel zu rotieren. Nutzen Sie für die Tastaturabfrage das Gerüst in der Eventloop und die Funktionalität von SDL3. Dokumentieren Sie welches Winkelvorgehen die Kamera in welche Richtung rotiert bzw. verschiebt. Erklären Sie warum die Kamera sich so verhält.

Lösung

-> Rechte-Hand-Regel (x Daumen zeigt in positive Richtung, y Zeigefinger zeigt in positive Richtung, z Mittelfinger zeigt in positive Richtung)

- Pitch Up: positive Rotation um die X-Achse
- Pitch Down: negative Rotation um die X-Achse -> Läuft man ins Positive auf der X-Achse, neigt sie sich nach oben
- Yaw Left: positive Rotation um die Y-Achse -> Läuft man ins Positive auf der Y-Achse, dreht man sich nach links
- Yaw Right: negative Rotation um die Y-Achse

Implementierung

- Statt switch-case wurde if-else verwendet, sodass mehrere Tasten gleichzeitig gedrückt werden können, sowie die Bewegung nicht durch den Hardware-Delay von Rising/ Falling-Edge beeinflusst wird.
- Für die Rotationen und Translationen wurden die internen Funktionen von `Ramen Camera.h` genutzt

3.7) Matrizen-Stack

Bauen Sie eine animierte Szene mithilfe eines Matrizenstapels und den von Ihnen erstellten primitiven Körpern. Implementieren Sie hierzu am besten eine Klasse, welche die Methoden aus der Vorlesung zur Verfügung stellt.

Lösung

- Es wurde eine Klasse *matrixStack.h* erstellt, die einen Stack (*vector*) von Matrizen verwaltet. Die Klasse besitzt Methoden zum Pushen und Popen von Matrizen sowie zum Multiplizieren der aktuellen Matrix mit einer neuen Transformation/ Rotation oder Skalierung.

Implementierung

- In der Rendering-Loop wird mit *matrixStack.Push()* eine neue Matrix auf den Stack gelegt und leitet damit die Transformationen für die neue Geometrie ein. Bei den Operationen der Multiplikation muss berücksichtigt werden, dass die Reihenfolge der Transformationen von rechts nach links (bzw. im Code von unten nach oben) gelesen werden muss.
- Dies ist besonders bei der Animation des Sonnensystems wichtig, da man für die Planeten zuerst die Translation für den Abstand zur Sonne machen muss und dann erst die Rotation um die Sonne.

Ergebnis

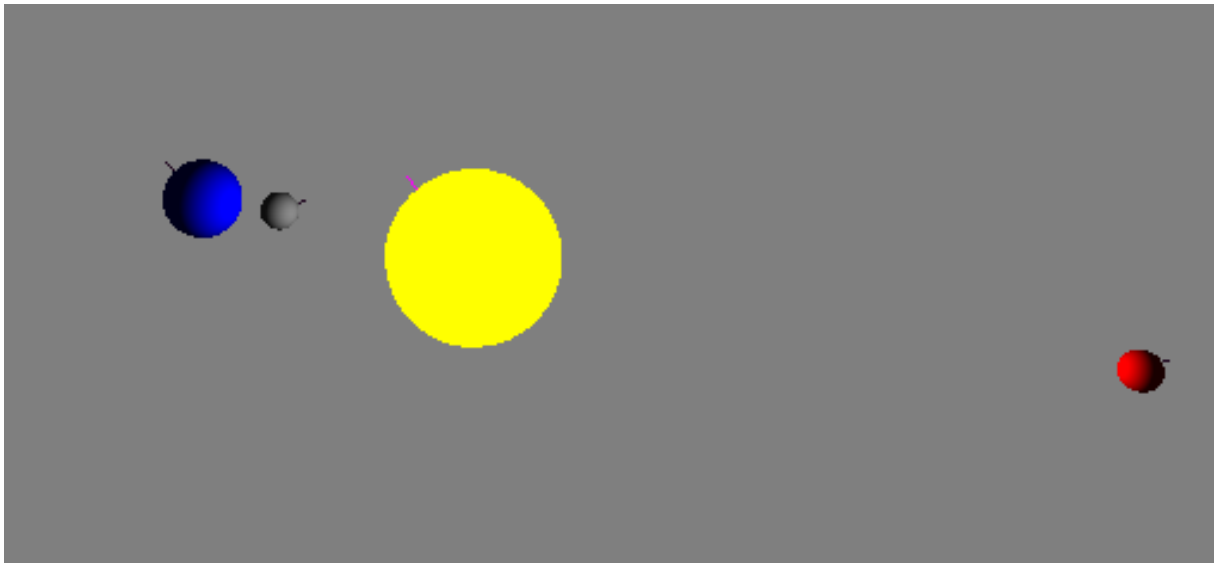


Figure 7: Animiertes Sonnensystem mit punktueller Lichtquelle (Sonne) und drei Himmelsobjekte, die sich um die Sonne drehen.